

MySQL CASE Statements

By mastering CASE statements, you can write more efficient and readable SQL queries, handling complex business logic directly in your database operations.

Example 2: Multiple Conditions

```
## Assign a salary category to each employee based on their salary range
SELECT employee_name,
CASE
    WHEN salary > 50000 THEN 'High Salary'
    WHEN salary BETWEEN 30000 AND 50000 THEN 'Medium Salary'
    ELSE 'Low Salary'
END AS salary_category
FROM employees;
```

Example 3: Using CASE in ORDER BY

CASE statements can also be used in ORDER BY clauses to create custom sorting logic:

```
## Categorize the quantities but also sort the results based on these categories
SELECT OrderID, Quantity,
CASE
    WHEN Quantity > 30 THEN 'The quantity is greater than 30'
    WHEN Quantity = 30 THEN 'The quantity is 30'
    ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails
ORDER BY
CASE
    WHEN Quantity > 30 THEN 1
    WHEN Quantity = 30 THEN 2
    ELSE 3
END;
```

Example 4: CASE in UPDATE Statements

CASE statements can be used in UPDATE statements to modify data conditionally:

```
## Assign a salary category to each employee based on their current salary
UPDATE employees
SET salary_category =
CASE
    WHEN salary > 50000 THEN 'High Salary'
    WHEN salary BETWEEN 30000 AND 50000 THEN 'Medium Salary'
    ELSE 'Low Salary'
END;
```

Best Practices

1. Always include an ELSE clause to handle unexpected cases.
2. Use CASE statements to simplify complex conditional logic in your queries.
3. Remember that CASE statements evaluate conditions in order and stop at the first true condition.
4. CASE statements can be used in SELECT, WHERE, ORDER BY, and UPDATE clauses, offering great flexibility in query design.